

Lekcja 05 - Wprowadzenie do Pythona

2026-08-13

Data Engineering Course

Wprowadzenie do Pythona dla Data Engineeringu

```
zmienne -> typy -> warunki -> petle -> list/dict  
-> funkcje -> CSV -> testy
```

Cel: napisac prosty program, ktory czyta zamowienia i liczy completed revenue.

■ Zaczynamy od srodowiska

W projekcie Python ustawiamy najpierw narzedzia:

```
pyenv -> wersja Pythona  
pipx -> instalacja narzedzi CLI  
uv -> szybkie install/run/tools dla Pythona  
Poetry -> zaleznosci i .venv projektu
```

Bez tego kazdy uczestnik moze miec inna wersje Pythona i inne paczki.

```
system Linux / WSL
  |
  v
pyenv: Python 3.13.0
  |
  v
pipx albo uv: narzedzia CLI
  |
  v
Poetry: .venv + pytest dla projektu
```

Jedna warstwa odpowiada za jedna rzecz.

■ Instalacja: Linux / WSL

Ubuntu / WSL Ubuntu:

```
sudo apt update
sudo apt install -y make build-essential
libssl-dev zlib1g-dev \
    libbz2-dev libreadline-dev libsqlite3-dev curl
git libffi-dev
curl https://pyenv.run | bash
```

Potem wpisujemy w `~/.zshrc` i nowy shell.

Pełna instrukcja: `student/setup.md`.

■ Instalacja: macOS i Windows

macOS:

```
brew install pyenv pipx uv  
pipx install poetry
```

Windows PowerShell:

```
winget install pyenv-win.pyenv-win  
python -m pip install --user pipx  
powershell -ExecutionPolicy ByPass -c "irm  
https://astral.sh/uv/install.ps1 | iex"  
pipx install poetry
```

Pełne kroki i debug są w [student/setup.md](#).

■ Python dla projektu

```
pyenv install 3.13.0  
pyenv global 3.13.0  
python --version
```

W katalogu projektu:

```
pyenv local 3.13.0  
cat .python-version
```

`.python-version` mówi: ten projekt używa tej wersji Pythona.

■ ppx, uv i Poetry

```
python -m pip install --user ppx
python -m ppx ensurepath
exec zsh
curl -Lsf https://astral.sh/uv/install.sh | sh
ppx install poetry
poetry --version
uv --version
```

Dlaczego **ppx** i **uv**?

ppx instaluje globalne narzędzia CLI w izolacji.
uv jest szybkim nowoczesnym narzędziem do Pythona.
Poetry prowadzi projekt tej lekcji.

■ Poetry w projekcie

```
poetry config virtualenvs.in-project true
mkdir -p homework/lesson_05
cd homework/lesson_05
pyenv local 3.13.0
poetry init --name lesson-05-python-intro --python
"^3.13" --no-interaction
poetry add --group dev pytest
```

Efekt:

```
.python-version
pyproject.toml
poetry.lock
.venv/
```

■ Uruchamianie przez Poetry

```
poetry run python python_basics.py  
poetry run pytest
```

Zapamiętaj:

```
python          -> może być przypadkowy  
interpreter  
poetry run python -> interpreter z venv tego  
projektu
```

W pracy domowej używamy `poetry run`.

■ Czego dziś nie robimy

Na tej lekcji nie wchodzimy jeszcze w:

```
duze biblioteki  
klasy  
architektury pipeline'ow  
orkiestracje  
cloud
```

Najpierw zwykły Python ma być jasny.

```
0-8      setup: pyenv, pipx, uv, Poetry
8-13     po co Python w Data Engineeringu
13-20    zmienne, typy, print
20-27    if / elif / else, wciecia
27-34    list, dict, petla for
34-40    funkcje, return, csv.DictReader
40-43    assert i pytest
43-45    homework contract
```

■ Po co Python w DE

SQL dobrze odpowiada na pytania w bazie.

Python dobrze opisuje proces:

```
wczytaj plik  
sprawdz rekordy  
policz wynik  
zapisz / wypisz rezultat  
uruchom to ponownie jutro
```

■ Najprostszy model programu

```
input data
  |
  v
instructions
  |
  v
output result
```

Program nie zgaduje intencji. Wykonuje dokładnie instrukcje, które napiszesz.

■ Pierwszy plik .py

```
print("hello Python")
```

Uruchomienie:

```
python student/lab/python_intro.py
```

`print` pokazuje cos w terminalu.

Zmienne

Zmienna to nazwa dla wartosci.

```
order_id = 1001
status = "completed"
total_amount = 120.50
is_paid = True
```

Pytanie:

Która z tych wartości jest tekstem, a która liczba?

■ Podstawowe typy

```
str    tekst, np. "completed"  
int    liczba całkowita, np. 1001  
float  liczba z przecinkiem, np. 120.50  
bool   True albo False
```

W Pythonie typ widzisz po wartości, nie po deklaracji jak w Javie.

■ Type check w głowie

```
amount_text = "120.50"  
amount_number = 120.50
```

To nie jest to samo.

```
"120.50"  tekst z CSV  
120.50    liczba do sumowania
```

■ First Principles: CSV daje tekst

CSV to plik tekstowy.

```
order_id,status,total_amount  
1001,completed,120.50
```

Po wczytaniu z CSV wartosc **120.50** zwykle przychodzi jako tekst.

Dlatego przed suma robimy konwersje.

Konwersja typow

```
amount_text = "120.50"  
amount = float(amount_text)
```

Teraz `amount` jest liczba.

```
revenue = 0.0  
revenue = revenue + amount
```

■ Warunek if

Warunek decyduje, która gałąź kodu się wykona.

```
status = "completed"

if status == "completed":
    print("liczymy revenue")
```

== pyta, czy wartości są równe.

■ if / elif / else

```
if status == "completed":  
    print("liczymy revenue")  
elif status == "cancelled":  
    print("pomijamy")  
else:  
    print("sprawdz recznie")
```

elif znaczy: jeśli poprzedni warunek nie przeszedł, sprawdź kolejny.

■ Wcięcia są składnia

Python nie używa `{ }` jak Java.

```
if status == "completed":  
    print("inside if")  
  
print("outside if")
```

To, co jest wcięciem pod `if`, należy do tej gałęzi.

■ Typowy blad: = vs ==

Zle:

```
if status = "completed":  
    print("ok")
```

Dobrze:

```
if status == "completed":  
    print("ok")
```

■ przypisuje. ■ porownuje.

■ Jeden rekord jako dict

Jeden order możemy opisać jako słownik.

```
order = {  
    "order_id": "1001",  
    "status": "completed",  
    "total_amount": "120.50",  
}
```

dict to struktura: klucz -> wartość.

■ Czytanie wartosci z dict

```
status = order["status"]  
amount_text = order["total_amount"]
```

Pytanie:

```
Co zwroci order["status"]?
```

Odpowiedz:

```
"completed"
```

Wiele rekordow jako list

```
orders = [  
    {"order_id": "1001", "status": "completed", "  
    total_amount": "120.50"},  
    {"order_id": "1002", "status": "cancelled", "  
    total_amount": "80.00"},  
]
```

list trzyma wiele elementow.

W tym case: wiele slownikow.

■ Petla for

Petla przechodzi po elementach listy.

```
for order in orders:  
    print(order["order_id"])
```

Czytaj to tak:

dla kazdego order w orders wykonaj blok kodu

■ Petla + warunek

```
for order in orders:  
    if order["status"] == "completed":  
        print(order["order_id"])
```

Pytanie:

Czy cancelled order zostanie wypisany?

Nie, bo nie przejdzie warunku.

■ Liczenie revenue

```
revenue = 0.0

for order in orders:
    if order["status"] == "completed":
        amount = float(order["total_amount"])
        revenue = revenue + amount
```

To jest najważniejszy mechanizm tej lekcji.

■ Checkpoint

Powiedz na głos:

```
orders to lista.  
order to słownik.  
for przechodzi po orderach.  
if wybiera completed orders.  
float zamienia tekst kwoty na liczbę.
```

■ Funkcja

Funkcja nazywa fragment logiki.

```
def is_completed(order: dict[str, str]) -> bool:  
    return order["status"] == "completed"
```

Nazwa funkcji mówi, co sprawdzamy.

■ return vs print

```
def add(a: int, b: int) -> int:  
    return a + b
```

return oddaje wynik programowi.

```
result = add(2, 3)
```

print tylko pokazuje wartosc czlowiekowi.

■ Funkcja do kwoty

```
def parse_amount(value: str) -> float:  
    return float(value)
```

Po co taka funkcja?

```
nazywa intencje  
upraszcza petle  
latwo ja testowac
```

■ Funkcja do revenue

```
def calculate_completed_revenue(orders:
list[dict[str, str]]) -> float:
    revenue = 0.0

    for order in orders:
        if is_completed(order):
            revenue = revenue + parse_amount
            (order["total_amount"])

    return revenue
```

■ Czytanie CSV

Używamy standardowej biblioteki.

```
import csv  
from pathlib import Path
```

Nie potrzebujemy tu jeszcze dużych bibliotek.

■ csv.DictReader

```
def read_orders(path: Path) -> list[dict[str,  
str]]:  
    with path.open(newline="") as file:  
        reader = csv.DictReader(file)  
        return list(reader)
```

DictReader zamienia wiersze CSV na słowniki.

■ CSV -> list[dict]

```
orders.csv  
|  
v  
csv.DictReader  
|  
v  
list[dict[str, str]]
```

Kazdy wiersz CSV staje sie jednym **dict**.

```
orders = read_orders(DATA_DIR / "orders.csv")

completed_revenue = calculate_completed_revenue(
    orders)
status_counts = count_orders_by_status(orders)
completed_order_ids = collect_completed_order_ids(
    orders)

print("Completed revenue:", completed_revenue)
```

■ Zliczanie statusow

```
counts: dict[str, int] = {}

for order in orders:
    status = order["status"]

    if status not in counts:
        counts[status] = 0

    counts[status] = counts[status] + 1
```

■ Dlaczego status normalizujemy

```
def normalize_status(status: str) -> str:  
    return status.strip().lower()
```

Przykłady:

```
" Completed " -> "completed"  
"COMPLETED" -> "completed"
```

■ Test przez assert

Najprostszy test:

```
assert normalize_status(" Completed ") == "  
completed"  
assert parse_amount("120.50") == 120.5
```

Jesli warunek jest falszywy, test pada.

Test to pytanie do kodu.

```
def test_cancelled_order_is_ignored() -> None:
    orders = [
        {"status": "completed", "total_amount": "
100.00"},
        {"status": "cancelled", "total_amount": "
50.00"},
    ]

    assert calculate_completed_revenue(orders) ==
100.0
```

■ Najważniejszy test tej lekcji

```
completed revenue liczy completed orders  
i ignoruje cancelled / refunded / pending
```

To sprawdza logikę biznesową, nie tylko składnię Pythona.

■ Typowe błędy

```
= zamiast ==  
brak wcięcia  
brak float przy kwocie z CSV  
print zamiast return  
jedna wielka funkcja bez nazwanych kroków
```

■ Co już umiemy zrobić

Na labie zbudowaliśmy małe klocki:

<code>normalize_status</code>	porządkuje tekst
<code>statusu</code>	
<code>is_completed</code>	sprawdza, czy order
<code>jest completed</code>	
<code>parse_amount</code>	zamienia tekst kwoty
<code>na liczbę</code>	
<code>calculate_completed_revenue</code>	sumuje kwoty
<code>completed orders</code>	
<code>read_orders</code>	czyta CSV do
<code>list[dict]</code>	

Homework to nie jest nowy temat.

Homework to samodzielna wersja tych samych klocków.

■ Co budujesz w homeworku

Budujesz jeden prosty program:

```
orders.csv
|
v
read_orders()
|
v
list[dict]
|
v
funkcje z petla for i if
|
v
wyniki w terminalu + testy
```

Cel programu:

```
policzyc metryki dla zamowien ze statusu completed
```

■ Jakie pliki oddajesz

W katalogu homeworku maja byc te pliki:

```
homework/lesson_05/  
├── python_basics.py      twoj kod  
├── test_python_basics.py pytania do kodu  
├── orders.csv           male dane wejsciowe  
└── notes.md             krotkie odpowiedzi  
słowami
```

W wersji z Poetry beda tez:

```
.python-version  
pyproject.toml  
poetry.lock
```

■ Co jest w `python_basics.py`

Ten plik ma mieć funkcje:

```
normalize_status(status)
is_completed(order)
parse_amount(value)
classify_amount(amount)
calculate_completed_revenue(orders)
count_orders_by_status(orders)
collect_completed_order_ids(orders)
read_orders(path)
main()
```

Czytaj to od góry:

```
najpierw male funkcje na jednej wartosci,
potem funkcje na liscie orders,
na koncu read_orders i main.
```

■ Co sprawdzają testy

Testy mają odpowiedzieć na proste pytania:

```
Czy " Completed " zamienia sie na "completed"?  
Czy cancelled order nie liczy sie do revenue?  
Czy "120.50" zamienia sie na 120.5?  
Czy statusy sa poprawnie zliczone?  
Czy lista completed IDs ma tylko completed orders?
```

Test nie jest dodatkiem.

Test pokazuje, że funkcja naprawdę robi to, co mówisz.

■ Jak poznasz, że jest dobrze

Z katalogu `homework/lesson_05` uruchamiasz:

```
poetry run python python_basics.py  
poetry run pytest
```

Oczekujesz dwóch rzeczy:

```
program wypisuje wyniki dla orders.csv  
testy przechodzą bez błędów
```

Jeśli test pada, czytaj komunikat:

```
która funkcja?  
jaka wartość oczekiwana?  
jaka wartość dostałem?
```

■ Closing check

Powiedz na głos:

```
orders.csv to plik tekstowy.  
csv.DictReader daje list[dict].  
Kazdy order jest dict.  
for przechodzi po orderach.  
if wybiera completed orders.  
float zamienia kwote z tekstu na liczbe.  
return oddaje wynik do testu.
```

■ Następna lekcja

Teraz rozumiesz mechanikę:

```
plik -> rekordy -> funkcje -> wynik -> testy
```

W lekcji 06 zrobimy z tego mały pipeline:

```
extract -> validate -> transform -> load
```